API Authentication Report

Prepared by: Charmaine Mangorima
Date: November 30, 2025
Platform: APISec University
Tools used: oauth.tools

Executive Summary

This report summarizes my learning experience and key takeaways from the API Authentication course offered by APIsec University. The course provided comprehensive coverage of modern authentication mechanisms for APIs, with particular emphasis on OAuth 2.0, JSON Web Tokens (JWT) and secure token management practices.

Throughout the course, I gained practical knowledge of how authentication systems work in real-world API environments, the roles different actors play in the authentication flow and best practices for implementing secure authentication in production systems.

Key Learning Outcomes:

- Deep understanding of OAuth 2.0 framework and major flows.
- Practical knowledge of JWT structure, lifecycle, and security considerations.
- Ability to implement scope-based authorization and claims management.
- Understanding of API gateway authentication patterns.
- Knowledge of token management and secure storage practices.

Module 1: API Authentication

OAuth 2.0 is a delegation protocol/framework that allows one application to access resources/APIs on behalf of a user without sharing the user's credentials. It does not rely on signing and it is easy to implement clients/apps using this protocol. OAuth requires HTTPS. It is also token based authorization and authentication. Example of where we use OAuth is when we use **"sign in with Google"** on a 3rd party app.

Module 2: OAuth Actors

The problem:

- The app/client needs to access the API on behalf of the user.
- The app should not see the user's credentials.
- The user needs to be aware that they allowed the App access.

OAuth Actors:

- **Resource Owner:** usually the user who owns the data
- **Client:** the app that wants to access the user's data. It requests access from the user and it only gets tokens not credentials.
- **Authorization Server:** the server that handles authentication and issues tokens. It confirms the user identity and issues authorization codes, access tokens, refresh tokens. Example **accounts.google.com** service **.**
- **Resource Server:** the server that stores and protects the user's data/APIs. It validates tokens from the client. Example Google Drive API, Facebook Graph API.

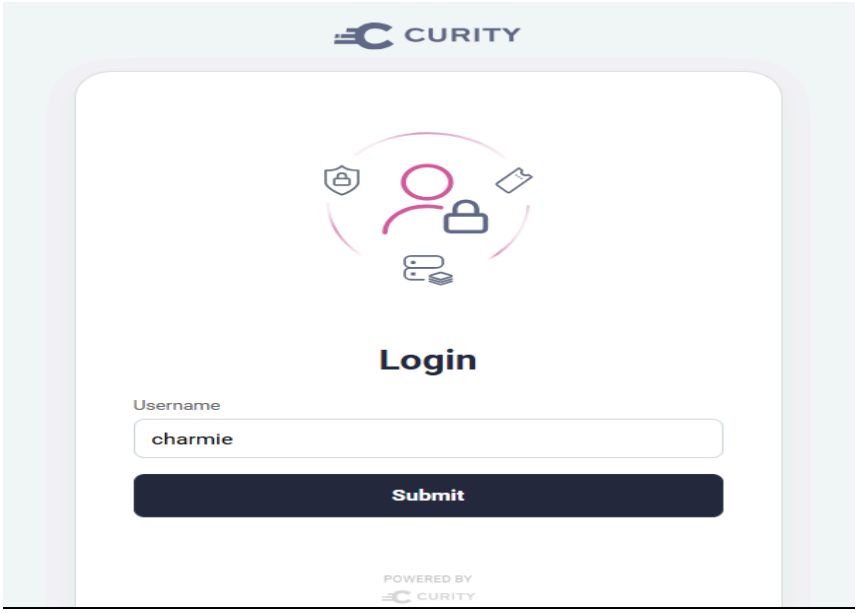
Module 3: OAuth Interaction Patterns(flows)

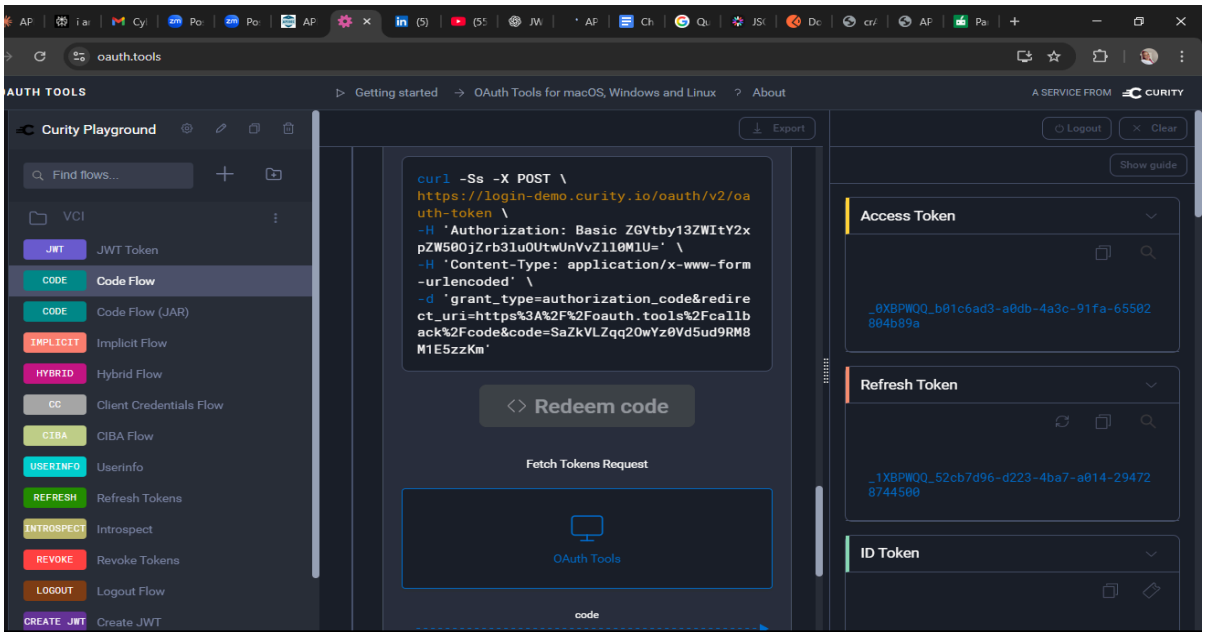
There are many types of OAuth flows, each suited for different scenarios:

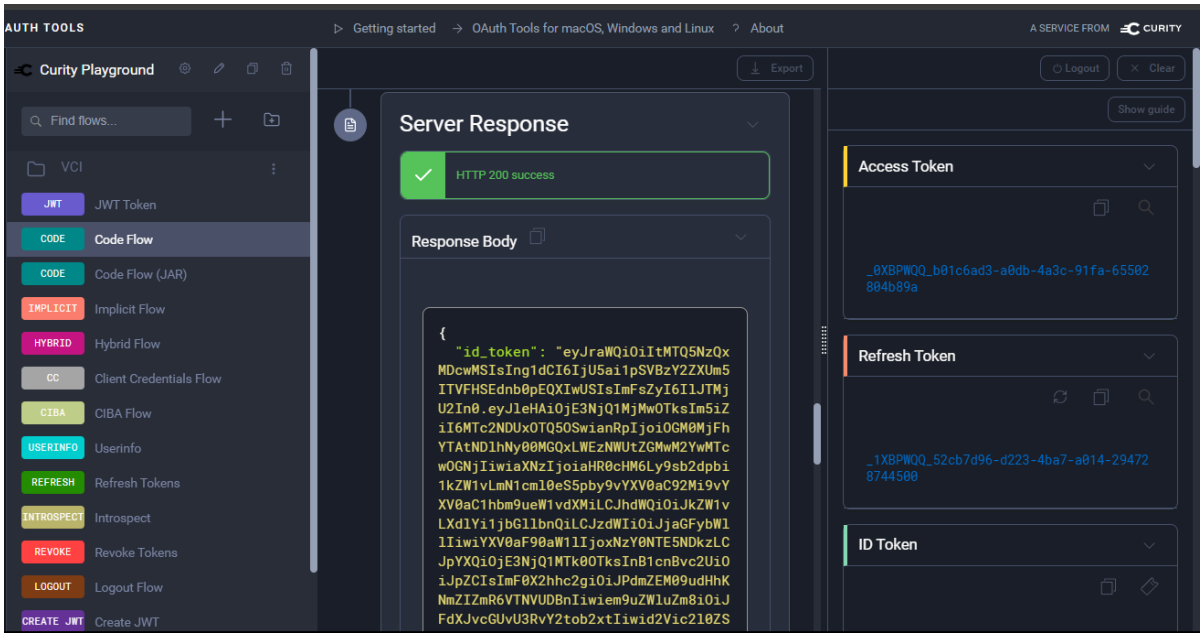
Authorization Code Flow

- Most secure flow for web and mobile applications
- Involves browser redirects and backend token exchange
- Best practice for applications that can securely store client secrets
- Returns access token and refresh token

  Demo on oauth.tools:

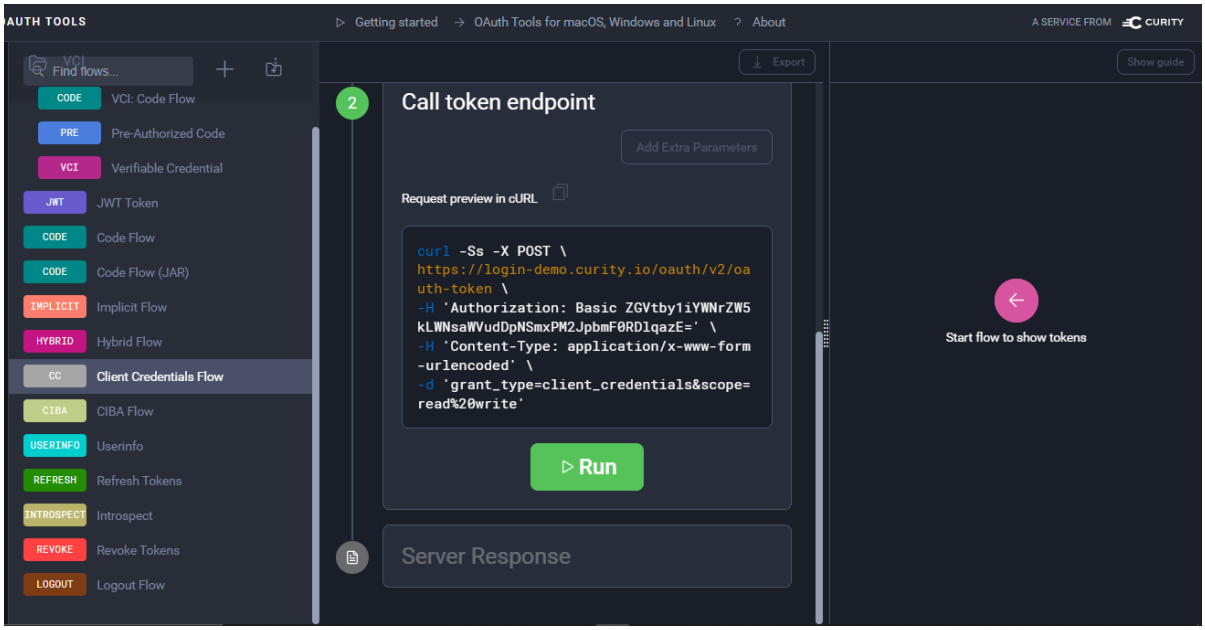

CURITY
Login
Username
charmie
Submit
POWERED BY
CURITY

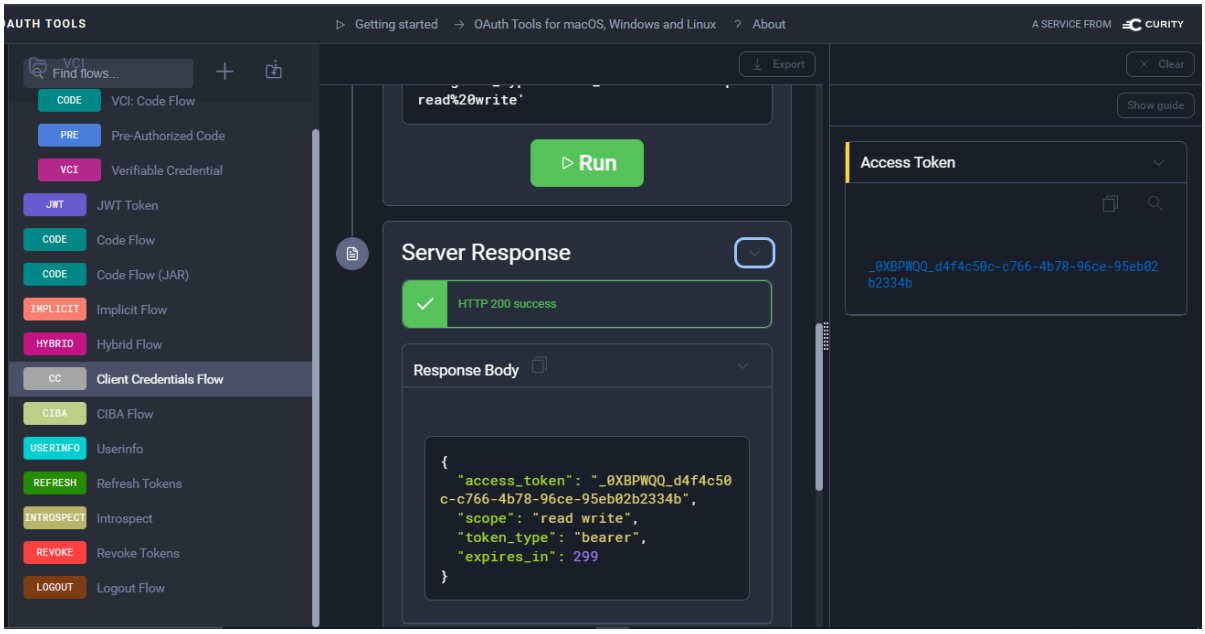




Client Credentials Flow

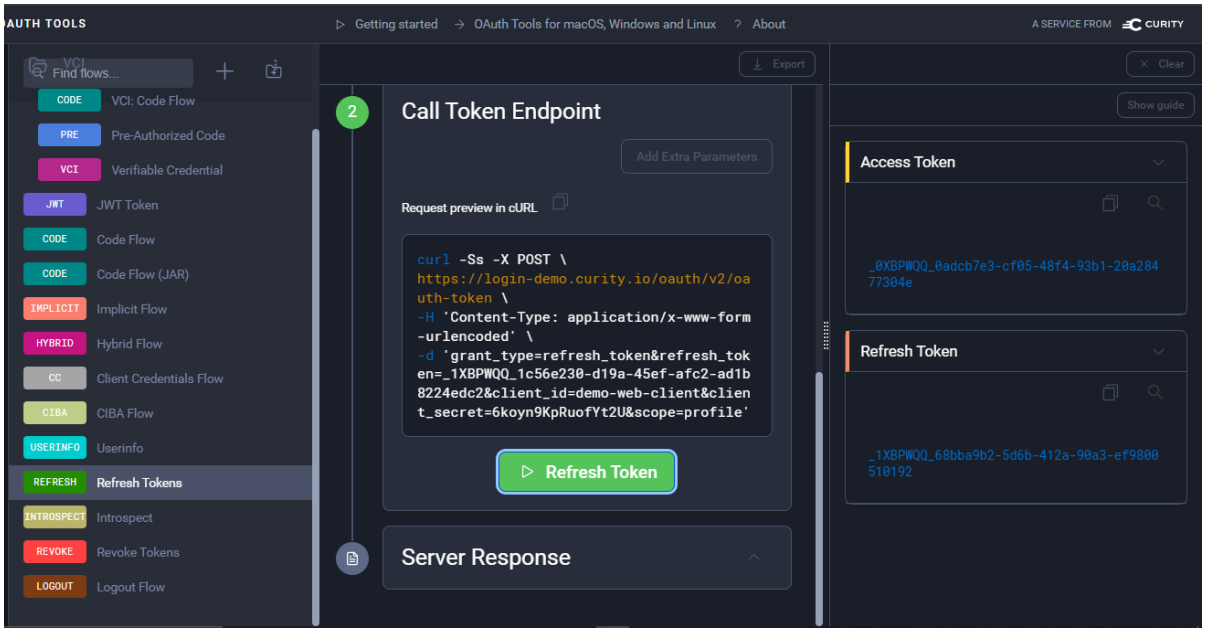
- For server-to-server authentication
- No user involvement, machine-to-machine communication
- Used for microservices and automated processes
- Returns access token only





Refresh Token Flow

- Used to obtain new access tokens without re-authentication of the user.
- Enables long-term access while maintaining security with short-lived access tokens.
- Refresh token is exchanged for a new access token when the current one expires.
- Critical for seamless user experience in mobile and web applications.



Module 4: Tokens

Formats: How to read its contents and how to validate it.

- Tokens are encoded either by value or by reference.
    1. **By reference:** it is opaque to the client- client cannot read it. It cannot be decrypted/decoded.
       This token points to an entry in the database that only the Authorization Server can decode.
       It requires a call to the Authorization server to read and validate and it is safer to use on the internet.
    2. **By value:** it is the opposite of by reference. Example JSON Web Tokens(JWTs)

Purpose: who is it for.

- **Access Tokens:** Meant for the Resource Server. They are short-lived tokens used to access protected resources. Typically valid for minutes to hours. Passed in HTTP Authorization header as Bearer tokens.

  GET /users/ HTTP1.1
  Host: api.example.com:8443
  Authorization: Bearer eJlk2gfff.gThhjkkg…….

- **Refresh Tokens:** Long-lived tokens used to obtain new access tokens. More sensitive and must be stored securely. Enable seamless user experience without repeated logins.

Type: how can they be used

- **Bearer Tokens:** can be used by anyone. If you have it then you can use it. More like cash, when you pick it you can use it anywhere without proving that you are the owner. They need to be protected, are not to be logged or to be kept in the browser.
- **Proof of Possession(POP) Tokens/Holder of Key(HOK):** These can not be sent or be used by other parties. They are more like a credit card, you need to prove ownership.

Module 5 :JWT (JSON Web Token)

It is a format(how we encode data as a token). It is used for ID tokens, access tokens. Refresh tokens are never JWTs.

the three-part structure of JWTs:

HEADER.PAYLOAD.SIGNATURE
Header: a few details about the token
Payload: actual content that is being protected
Signature: signs the token

Header:
```
{
  "alg": "HS256",
  "typ": "JWT"
}
```
Specifies the signing algorithm
Declares the token type

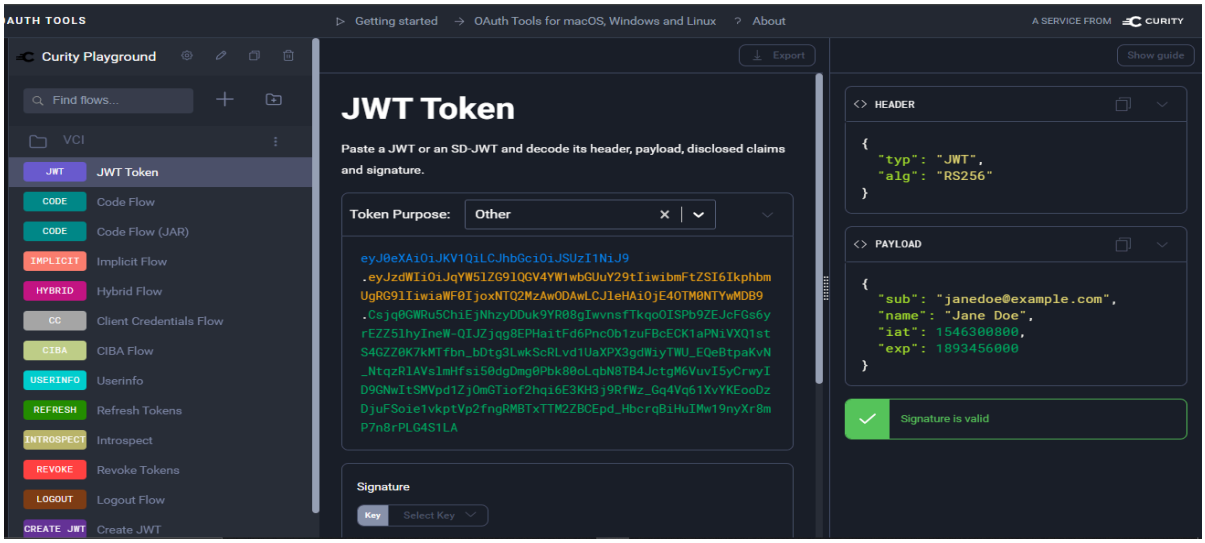
Payload:
```
{
  "sub": "user123",
  "name": "John Doe",
  "role": "admin",
  "exp": 1735689600
}
```
Contains claims (user information)
Includes standard claims like “sub” (subject), “exp” (expiration)
Can include custom claims for application needs

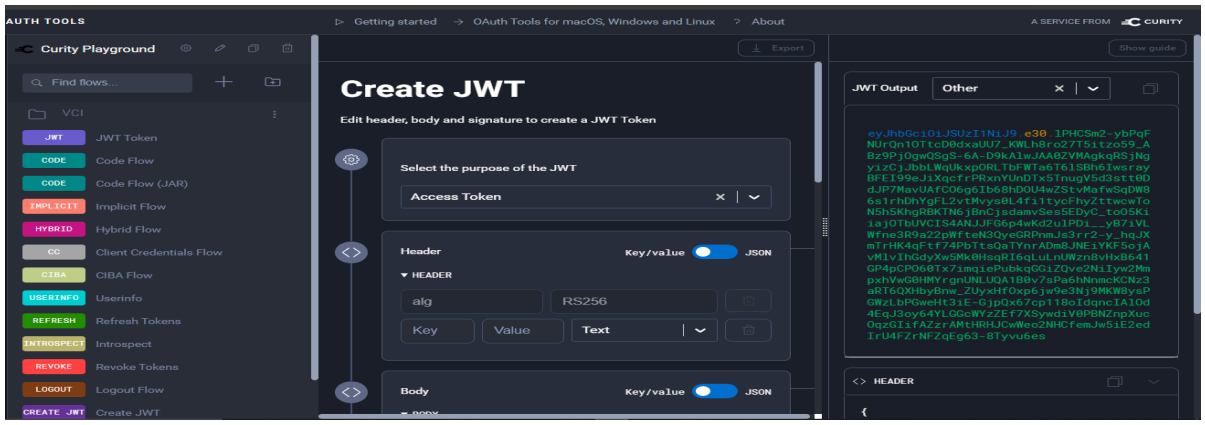
Signature:

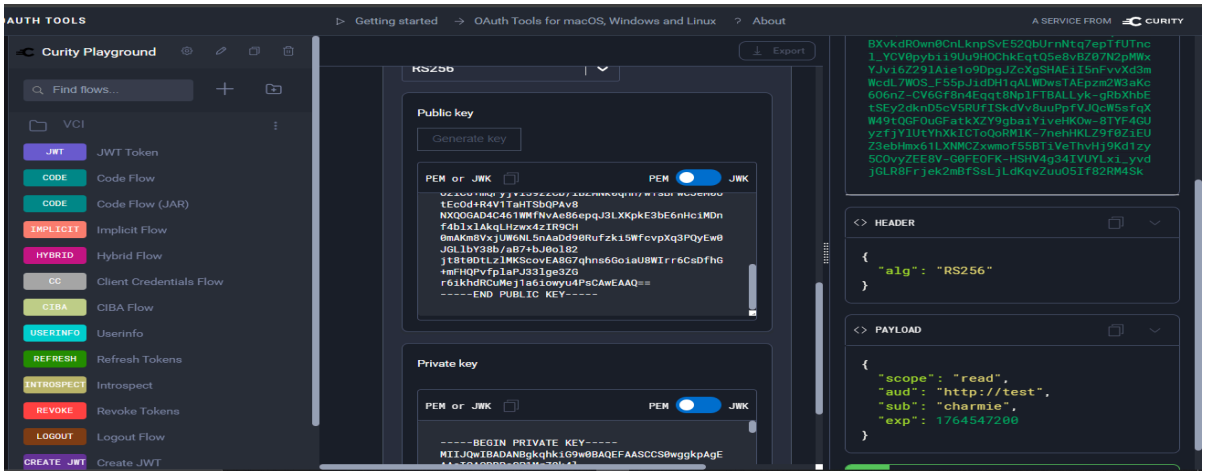
Cryptographic signature ensuring token integrity

Prevents tampering with token contents

Verifies token authenticity







- JWTs are self-contained and stateless, reducing database lookups.
- Tokens should have short expiration times to limit exposure.
- The signature prevents attackers from modifying token contents.
- Refresh tokens enable better security through token rotation.

Module 6: Scopes and Claims

Scopes define the permissions or access levels that a token grants. They are requested by the client, consented by the user and authorized by the Authorization Server.

Examples:
read_profile: read user profile information
write_posts: create or modify posts
delete_account: delete user account
user_invoice_update: update invoice

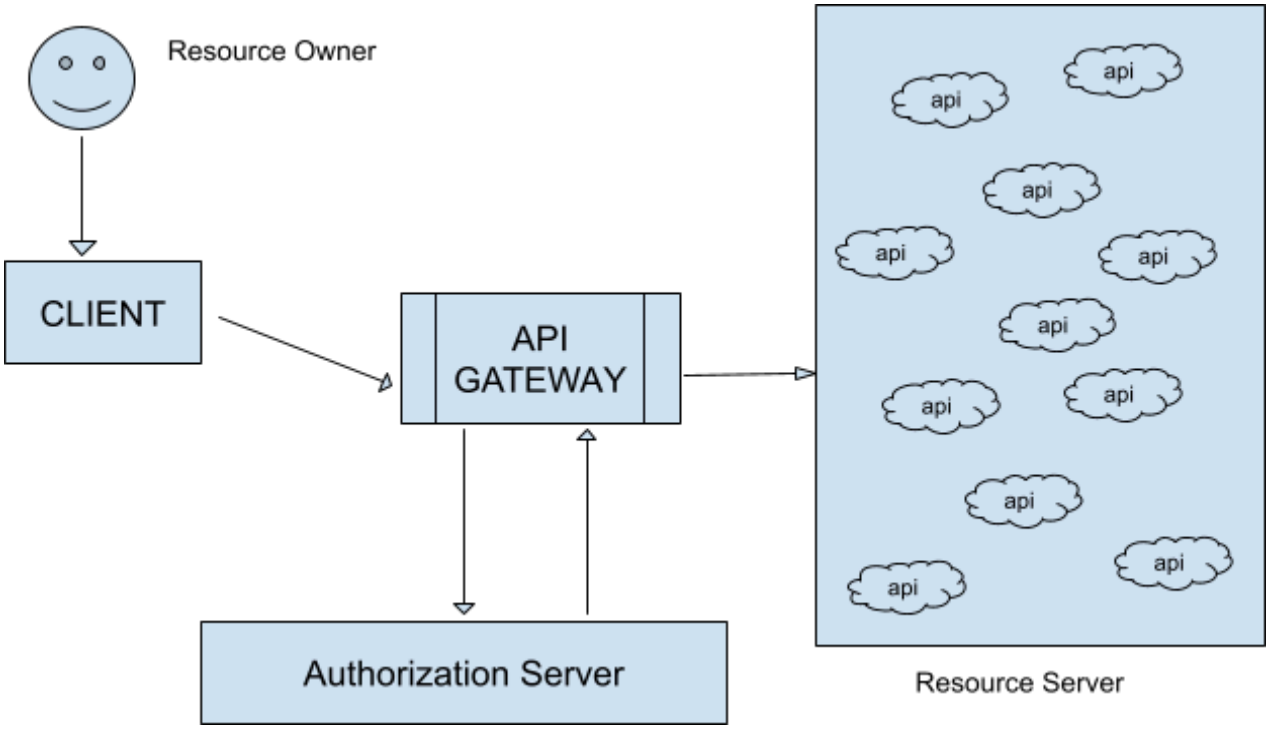
When writing scopes, use the least privilege principle
- Request only the minimum scopes needed
- Reduces risk if tokens are compromised
- Improves user trust by limiting permissions

Claims are statements(key-value pairs) about the user or entity contained within tokens:

Examples:
sub (subject) - Unique user identifier
iss (issuer) - Who issued the token
aud (audience) - Who the token is intended for
exp (expiration) - When token expires
iat (issued at) - When token was created
nbf (not before) - Token not valid before this time
Custom Claims: email, name, role, department ,organization

Data to put in a token:
- Should be relevant to a large set of your APIs
- Should not be application specific
- Should be attributes of the user
- Should not be contextual for the session

Module 7: APIs and Gateways



API Gateway Functions:

Authentication Enforcement:

- Centralized authentication checks before requests reach backend services
- Token validation and signature verification
- Single point of authentication policy enforcement

Token Management:

- Token introspection and validation

Request Routing:

- Route authenticated requests to appropriate microservices/APIs in the Resource Server
-

The gateway checks for scopes(course-grained authorization) and the API/microservice checks for claims for the particular scope(fine-grained authorization).

Conclusion

With a strong emphasis on JSON Web Tokens (JWTs) and the OAuth 2.0 framework, the API Authentication course offered thorough discussion of authentication methods for

APIs. I developed a deep comprehension of the header.payload.signature structure of JWT and how cryptographic signatures enable stateless authentication while preventing manipulation. Short-lived access tokens reduce security risk, but long-lived refresh tokens provide a smooth user experience. I got a clear understanding of the four actors of OAuth 2.0 namely Resource Owner, Client, Resource Server and Authorization Server, as well as the numerous flows for various scenarios such as Authorization Code, Client Credentials and Refresh Token flows.

End of report